CI/CD / DEVOPS / OPERATIONS

Why Quality Code Matters and How To Achieve It

Avoiding technical debt isn't merely about writing "good" code; it's a continuous commitment to clarity, maintainability and collaboration.

Jul 2nd, 2025 9:00am by Liz Acosta

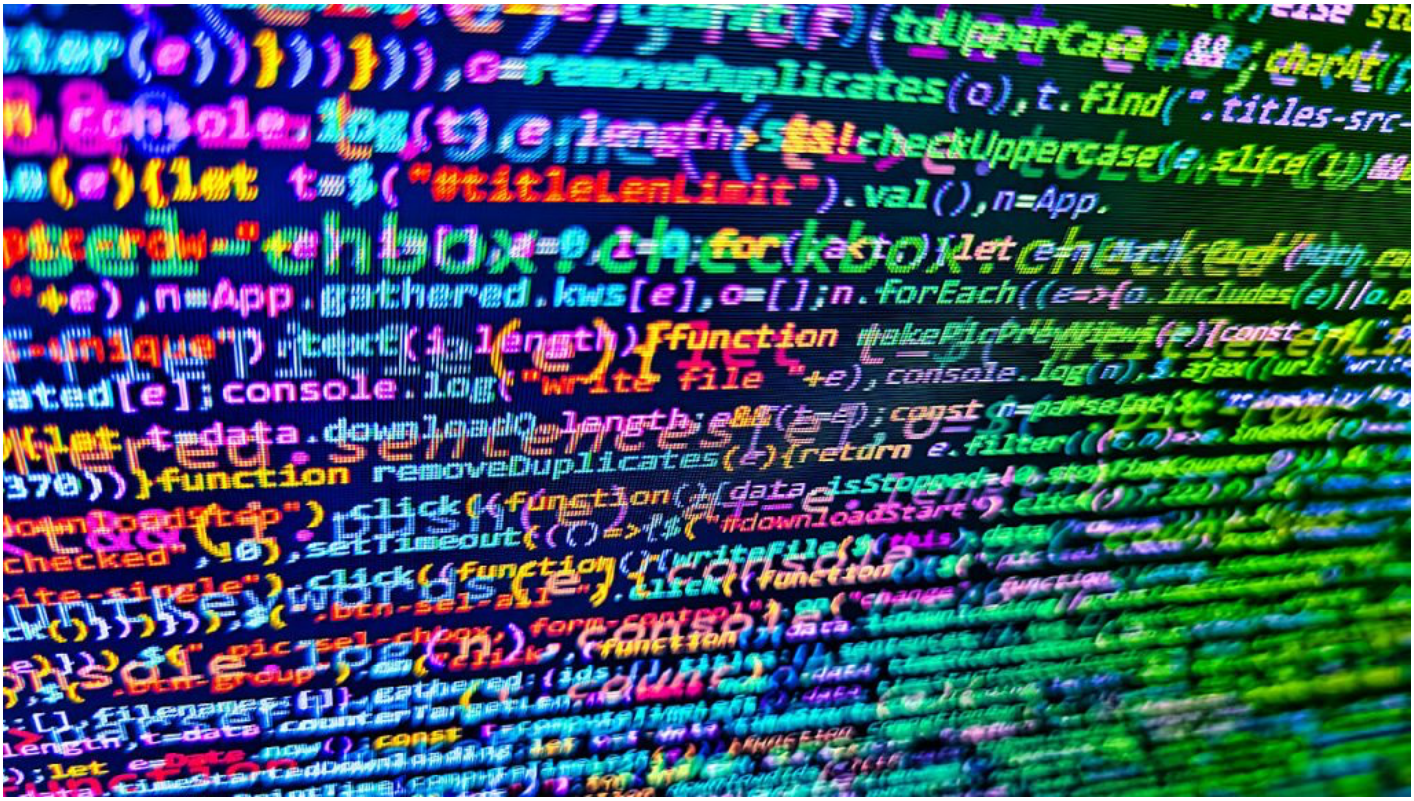

Image from BEST-BACKGROUNDS on Shutterstock.

frustrated development teams. With the increasing use of AI-generated code, the potential for introducing problematic code into software systems is growing. Regardless of how the software development landscape evolves, consistently applied best practices will always be crucial for maintaining code quality.

Let's explore the concept of technical debt, differences between good and bad code, essential best practices and practical strategies for integrating them into your development workflow.

The Cost of Neglecting Code Quality

The phrase "we'll add that to the backlog" often foreshadows the accumulation of technical debt. This refers to the extra work required in the future to rectify the consequences of quicker, more expedient solutions taken in the present, such as architectural shortcuts or rushed development. Much like financial debt, tech debt accrues "interest" over time, becoming increasingly expensive to address.

Consider the incident in January 2023 when over 1,300 flights in the U.S. were canceled and 10,000 more delayed. The cause? Accidentally deleted files during a database update that crippled the Notice to Air Missions (NOTAM) system. This wasn't an isolated event; neglecting tech debt had been an ongoing issue for years.

Essentially, tech debt stems from unaddressed

SHARE THIS STORY

TRENDING STORIES

1. This simple infrastructure gap is holding back AI productivity
2. How Adidas Manages for Scale
3. Spec-Driven Development: The Key to Scalable AI Agents
4. QCon chat: Is agentic AI killing continuous integration?
5. A Primer: Continuous Integration and Continuous Delivery (CI/CD)

technical decisions, including poorly written code, that proliferate throughout a software system, leading to future complications. When these fragile systems eventually collapse, the cost of repair far exceeds what it would have cost to prevent the debt in the first place. Moreover, developers spend, on average, 23% of their time on technical debt, leading to cognitive strain, dissatisfaction, burnout and even more debt.

The seemingly simple solution is to avoid pushing bad code. However, daily deadlines, the complexities of existing systems and the demand for quick fixes often make this a considerable challenge. Consistently writing clear, maintainable code demands discipline, awareness and the implementation of robust development practices.

The Good, the Bad, and the Ugly: Defining 'Good Code'

What constitutes "bad code?" It encompasses many things: hardcoded secrets, overly complex functions, missing unit tests, unnecessary repetition, unclear documentation and more. While some bad code is simply difficult to understand, other forms might be easy to grasp but still suffer from inefficiency, lack of maintainability or poor design. Code that's harder to comprehend is more susceptible to bugs and vulnerabilities because its complexity makes it challenging for developers to correctly interpret its logic, leading to errors during modification, debugging or integration.

TRENDING STORIES

1. This simple infrastructure gap is holding back AI productivity
2. How Adidas Manages for Scale
3. Spec-Driven Development: The Key to Scalable AI Agents
4. QCon chat: Is agentic AI killing continuous integration?
5. A Primer: Continuous Integration and Continuous Delivery (CI/CD)

If you find yourself repeatedly rereading a section of code, it's likely a candidate for refactoring. Difficulty in code comprehension isn't just an inconvenience; it significantly complicates debugging, especially during critical outages. That's when code truly becomes "ugly."

So, what is "good code?" While readability is a component, defining good code solely by this characteristic is an oversimplification. Good code also embraces factors like maintainability, efficiency, clarity of intent and adherence to best practices. The specific attributes of good code can vary based on programming languages, system architectures and frameworks. However, fundamentally, high-quality code is maintainable, reliable and secure.

Best Practices for Code Quality in Any Language

- **Never hardcode secrets:** Hardcoding sensitive information directly into your code is a significant source of technical debt and security risk. In

October 2022, Toyota discovered a hardcoded secret exposed in a public repository for nearly five years. Sensitive information like API tokens should always be retrieved from secure environments, not embedded in the code itself.

- **Reduce redundancy by embracing the DRY principle:** The "don't repeat yourself" (DRY) principle is fundamental. Reducing redundancy minimizes vulnerabilities and makes code easier to maintain. Strive to identify and eliminate duplication wherever possible.
- **Define, divide and decouple responsibilities for maintainability:** The single responsibility principle (SRP) advocates for simplifying code by narrowing down classes and modules to a single task. This principle extends to the way we define, divide and decouple services and architectural components, ultimately mitigating tech debt. Overcomplicated functions with multiple responsibilities become harder to read, test and maintain. By breaking tasks into separate, focused functions, you define and decouple them, resulting in code that's easier to understand, test, maintain and extend.
- **Write effective code comments and documentation:** Meaningful comments enhance code readability, but excessive or poorly written ones can hinder it. Use comments judiciously to explain the "why" behind the code, not just the "what." Meaningful names for variables and functions often reduce the need for extensive commenting. Additionally, use docstrings to document the purpose, arguments and return values of functions and classes.
- **Maintain consistency and build a lasting code legacy:** Enforcing a unified style throughout your codebase is the easiest way to maintain consistency, including choices like indentation

and naming conventions. A consistent code style streamlines the adoption of best practices, leading to consistency in unit testing standards, safer refactoring and better documentation. It also helps with onboarding new team members.

From Theory To Practice: Implementing Coding Standards

Understanding best practices is one thing; implementing them is another. Remember that best practices are guidelines designed to empower developers to work collaboratively and efficiently, not to control them. They should inspire and enable, not limit.

- **Learn from others:** Actively read code from reputable open source projects to gain insights into different styles, perspectives and use cases.
- **Leverage automation and shift left:** Integrate static code analysis tools, linters and code reviews into your development workflow as early as possible. These tools can automatically identify potential issues and enforce coding standards, minimizing the impact of bad code and enabling focus on more complex problems.
- **Cultivate curiosity and embrace exceptions:** Understand the rationale behind coding style enforcements. Occasionally, there might be valid reasons to deviate from a standard, but these exceptions should be conscious and well-documented.

Avoiding technical debt isn't merely about writing "good" code; it's a continuous commitment to clarity, maintainability and collaboration. By embracing best practices, you can build more robust, scalable and

sustainable software systems. Additionally, enforcing code quality at scale with tools like SonarQube that integrate seamlessly with existing development pipelines helps make this easier.

The effort invested in writing good code today pays significant dividends in reduced maintenance costs, fewer outages and a more productive and satisfied development team tomorrow.

TNS

Liz Acosta is a developer advocate at Sonar. A film student turned social media manager and content creator turned engineer turned developer advocate, she loves pizza, plants, pugs and Python. She is particularly interested in the intersection of tech and...

Read more from Liz Acosta →

ARCHITECTURE

Cloud Native Ecosystem
Containers
Databases
Edge Computing
Infrastructure as Code
Linux
Microservices
Open Source
Networking
Storage

ENGINEERING

AI
AI Engineering
API Management
Backend development
Data
Frontend Development
Large Language Models
Security
Software Development
WebAssembly

OPERATIONS

AI Operations
CI/CD
Cloud Services
DevOps
Kubernetes
Observability
Operations
Platform Engineering

CHANNELS

Podcasts
Ebooks
Events
Webinars
Newsletter
TNS RSS Feeds

THE NEW STACK

About / Contact
Sponsors
Advertise With Us
Contributions

roadmap.sh

Community created roadmaps, articles, resources and journeys for developers to help you choose your path and grow in your career.

Frontend Developer Roadmap
Backend Developer Roadmap
Devops Roadmap

© The New Stack 2026

Disclosures Terms of Use Advertising Terms & Conditions Privacy Policy Cookie Policy

FOLLOW TNS